Instruction Count Analysis Across ISAs

P Mahitha
Roll No: CS24BT054

January 19, 2026

```c
#include <stdio.h>
int main(){
    int x;
    printf("enter an integer:%d");
    scanf("%d",&x);
    printf("integer:%d\n",x);
    printf("square of the integer:%d",x*x);
    return 0;
}
```

Figure 1: Q1 C Code

```c
#include<stdio.h>
int main(){
    int a;
    printf("enter a positive integer");
    scanf("%d",&a);
    for(int i=a-1;i>=2;i--){
       a = a * i;
    }
    printf("factorial : %d ",a);
    return 0;
}
```

Figure 2: Q2 C Code

ISA and Compiler	Q1	Q2	Q3	Q4	Q5
RISC-V 32-bits gcc 14.2.0	30	37	70	30	71
RISC-V 64-bits gcc (trunk)	31	39	73	32	79
RISC-V rv32gc gcc (trunk)	NA	NA	NA	NA	NA
RISC-V rv32gc clang (trunk)	28	39	75	34	74
RISC-V rv64gc clang (trunk)	28	39	75	35	74
x86-64 clang 12.0.0	24	31	48	23	51
x86-64 gcc 14.2.0	26	29	49	21	56
ARM64 gcc 14.2.0	25	32	54	26	57
ARM32 gcc 14.2.0	28	35	69	33	66
ARMv8-a clang 19.1.0	27	36	57	29	60
MIPS64 gcc 5.4	48	55	82	35	97

a. For the same program, different ISAs generate different numbers of instructions because each architecture has different instruction sets and capabilities. Some ISAs need more instructions to do the same work.

b. Even for the same ISA, different compilers generate different (but not with so huge difference) instruction sequences because they use different optimization and code-generation techniques.

c. For the same ISA and compiler, 64-bit machines usually generate slightly more instructions due to handling larger data sizes and addresses.